A TWO-PART LECTURE

From Perceptrons to Sigmoid Neurons

How a single artificial neuron thinks, learns, and hits its limits

This lecture builds one neuron from nothing, in Python, twice -- first as a perceptron, then as its smoother cousin, the sigmoid neuron. Each page pairs a short piece of math with the exact code it becomes, so the equation and the program are never more than a paragraph apart.

PART I -- Perceptrons

1 What is a perceptron?

2 The equation, and a hand-worked example

3 Seeing it geometrically -- the decision boundary

4 Setting up the code

5 Coding the forward pass

6 The learning rule, in plain terms

7 Coding the training loop

8 Learning NAND from scratch

9 Reading the results

10 Where it breaks: XOR

PART II -- Sigmoid Neurons

11 Why a smoother neuron at all

12 The sigmoid equation

13 Seeing it -- step vs. sigmoid, side by side

14 Coding the forward pass

15 The slope, and why it matters

16 Coding the derivative

17 The gradient descent update rule

18 Coding the training loop

19 Learning NAND, with real numbers

20 What smoothness doesn't fix

21 Where this goes next

What Is a Perceptron?

Strip away the terminology and a perceptron is just a small weighing machine. It takes in a handful of numbers, decides how much each one matters, adds them up, and gives you a single yes or no at the end.

That's genuinely the whole idea -- no hidden step, no trick. Every input gets scaled by a weight that reflects its importance, the scaled values get summed, and if that sum clears a certain bar, the perceptron fires and returns 1. If it doesn't clear the bar, it returns 0.

Three moving parts

Inputs (x) are the raw numbers coming in.
Weights (w), one per input, get learned over time and control how much each input should matter.
Bias (b) is a single extra number that shifts how easy or hard it is for the whole thing to fire.

Every page that follows is just this idea, turned into working code: represent x and w as arrays, compute the weighted sum, compare it against zero, and write a rule that nudges w and b whenever the perceptron gets something wrong.

The Equation, Worked by Hand

Before any code, here's the math in its final form -- because this compact version is exactly what the code ends up computing.

$$z = w \cdot x + b$$

$$\text{output} = 1 \text{ if } z > 0 \text{ else } 0$$

w·x is the dot product -- multiply each weight by its matching input, then add the products together. For two inputs, that's just w1·x1 + w2·x2. The dot product notation only exists so the formula doesn't grow longer every time another input joins the list.

b, the bias, gets added afterward and doesn't depend on any input at all -- it's a constant nudge that makes the whole unit either more eager or more reluctant to fire.

A hand-worked example

Let w = (-2, -2), b = 3, and x = (1, 0):

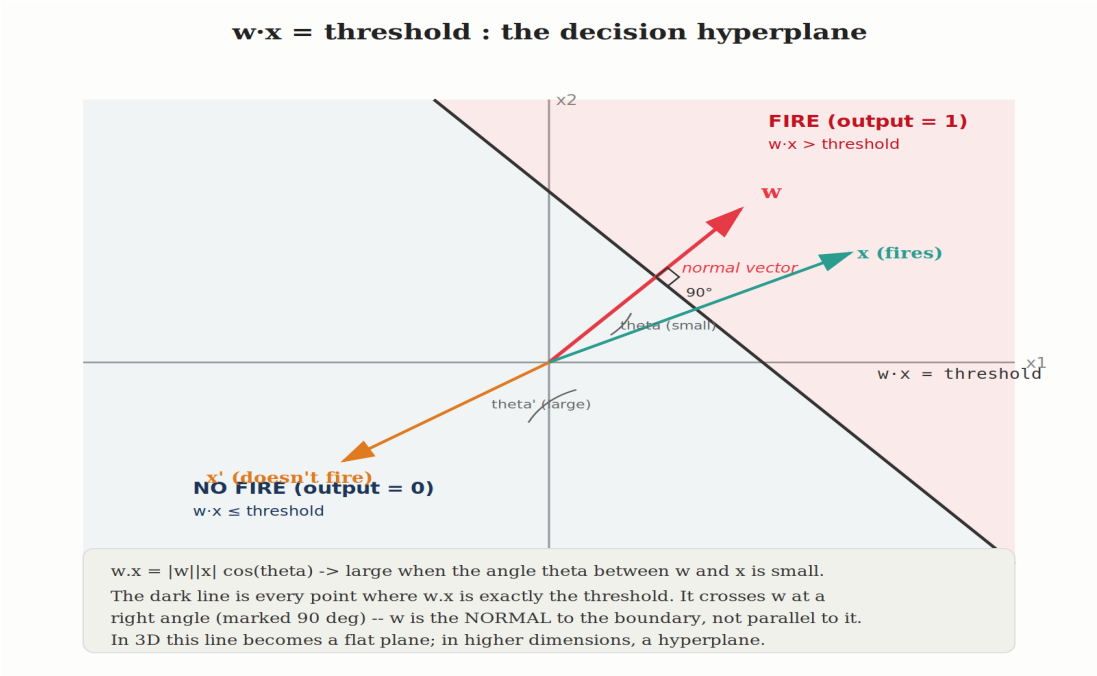
$$z = (-2)(1) + (-2)(0) + 3 = 1$$

$$1 > 0 \rightarrow \text{fires (output = 1)}$$

That's the entire computation, done by hand. Page 4 turns it into three lines of Python.

Seeing It: The Decision Boundary

The equation says a perceptron fires when w·x is bigger than the threshold. Geometrically, that's a much simpler statement than it sounds: w is an arrow, and everything the arrow points toward fires. Everything on the other side doesn't.



w acts as the normal vector to the decision boundary. The boundary always sits perpendicular to w -- never parallel to it.

The dark line is every point where w·x lands exactly on the threshold -- neither firing nor not firing, the knife-edge case. It always crosses w at a right angle, which is why w is called the 'normal vector' of the boundary. Move in the direction w points, and you're moving toward firing. Move against it, and you're moving away.

The one fact worth carrying forward from this page: a single perceptron can only ever draw one straight line through its input space. That single fact is what page 10 breaks the whole model on.

Setting Up the Code

In code, w and x become NumPy arrays -- NumPy can multiply and sum whole arrays in a single call, which lines up almost word for word with the dot product notation from page 2.

The perceptron is built as a small class: one object that bundles its own data (weights, bias) together with the actions that use that data (predict, train). Grouping state and behaviour like this is a pattern you'll see constantly in real software, not just here.

```
import numpy as np

class Perceptron:
    def __init__(self, num_inputs, learning_rate=0.1):
        # Small random starting weights, zero bias.
        # Both get adjusted once training begins.
        self.weights = np.random.randn(num_inputs) * 0.1
        self.bias = 0.0
        self.learning_rate = learning_rate
```

__init__ runs once, the moment a Perceptron object is created. It doesn't compute anything -- it just sets up the starting values that predict() and train() will read and later update.

Coding the Forward Pass

The 'forward pass' is the arithmetic from page 2, written as a method. Feed it an input, and it hands back a decision.

```
    def predict(self, x):
        """z = w.x + b, then the step function."""
        z = np.dot(self.weights, x) + self.bias
        return 1 if z > 0 else 0
```

Reading it line by line

np.dot(self.weights, x) computes w·x in one call -- the entire weighted sum, done for you. Adding self.bias matches b in the equation exactly. And return 1 if z > 0 else 0 is the step function -- the hard yes-or-no decision that makes this a perceptron rather than plain linear regression.

At this point the perceptron can already make decisions. It just doesn't know good weights yet -- that's what training is for.

The Learning Rule, in Plain Terms

Training means adjusting w and b until predictions match reality. The rule only reacts when something goes wrong:

```
w  =  w + (learning_rate) × (y - y_hat) × x

b  =  b + (learning_rate) × (y - y_hat)
```

y is the correct answer. y_hat is what the perceptron actually guessed. When they match, (y - y_hat) is zero and nothing moves at all -- a correct guess teaches the model nothing new, by design.

When the perceptron should have fired but didn't, the error is +1 and the weights shift toward x, making that mistake less likely next time. When it fired but shouldn't have, the error is -1 and the weights shift away from x instead. The learning_rate is just how big each of those corrections is allowed to be -- large enough to actually learn, small enough not to overcorrect on a single mistake.

Coding the Training Loop

The rule from page 6, applied over every example, repeated for several passes through the data.

```
    def train(self, training_inputs, labels, epochs=20):
        for epoch in range(epochs):
            total_errors = 0
            for x, y in zip(training_inputs, labels):
                x = np.array(x)
                y_hat = self.predict(x)
                error = y - y_hat

                if error != 0:
                    self.weights += self.learning_rate * error * x
                    self.bias += self.learning_rate * error
                    total_errors += 1

            if total_errors == 0:
                print(f"Converged after {epoch + 1} epochs.")
                break
```

The outer loop repeats the full dataset multiple times, since one pass usually isn't enough. The inner loop walks through each example, comparing prediction to truth. The update only happens inside if error != 0 -- correct guesses are skipped entirely. And once a full pass produces zero mistakes, the loop breaks early: the perceptron has converged, and further training would just be wasted effort.

Learning NAND From Scratch

With predict() and train() both written, the perceptron can now discover its own weights from raw examples, rather than being handed them the way page 2 did.

```
inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]
labels = [1, 1, 1, 0]          # the NAND truth table

p = Perceptron(num_inputs=2)
p.train(inputs, labels, epochs=50)

for x, y in zip(inputs, labels):
    prediction = p.predict(np.array(x))
    print(x, "-> predicted:", prediction, " expected:", y)
```

The training data here is nothing more than the NAND truth table. The perceptron is never told the rule for NAND -- it only sees examples of inputs paired with correct outputs, and the training loop searches for weights that satisfy all four at once.

Reading the Results

Running the code on the previous page, starting from real random weights, produces this:

```
Converged after 6 epochs.
Learned weights: [-0.18 -0.07]   bias: 0.20

x1  x2 | expected | predicted
 0   0 |        1 |         1   (correct)
 0   1 |        1 |         1   (correct)
 1   0 |        1 |         1   (correct)
 1   1 |        0 |         0   (correct)
```

A couple of things worth noticing. The learned weights here (-0.18, -0.07, bias 0.20) don't match the hand-picked ones from page 2 (-2, -2, bias 3) -- and yet both are correct. There's no single right answer for w and b, only a whole region of values that happen to separate the 1s from the 0s.

The bias also came out positive, which fits: NAND fires for three of four combinations, so the training process needed a bias that makes firing the default, switched off only when both inputs are strongly on.

Where It Breaks: XOR

Swap NAND's labels for XOR's and run the identical code -- nothing about predict() or train() changes:

```
labels = [0, 1, 1, 0]   # XOR truth table
p = Perceptron(num_inputs=2)
p.train(inputs, labels, epochs=50)
```

No 'Converged' message ever prints, no matter how many epochs you allow:

```
x1  x2 | expected | predicted
 0   0 |        0 |        1   (WRONG)
 0   1 |        1 |        1   (correct)
 1   0 |        1 |        0   (WRONG)
 1   1 |        0 |        0   (correct)
```

This isn't a bug, and it can't be fixed by tuning the learning rate or training for longer. Page 3 already showed why: a perceptron's decision boundary is always one straight line. XOR's two 'fire' points and two 'don't fire' points sit diagonally across from each other, and no single straight line can ever separate them. This geometric limit, not a coding mistake, is the entire reason multi-layer networks exist.

Why a Smoother Neuron at All

A sigmoid neuron keeps everything from Part I -- same inputs, same weights, same bias, same weighted sum $z = w \cdot x + b$. Only the very last step changes.

A perceptron takes z and slams it through a step function: 0 or 1, nothing between. A sigmoid neuron passes z through a smooth S-shaped curve instead, producing any value between 0 and 1 -- 0.5, 0.83, 0.02, whatever the math says.

Why bother?

The perceptron's learning rule from page 6 only fires on outright mistakes -- there's no notion of 'how wrong', only right or wrong. And a step function has no slope anywhere except one undefined point, so there's no way to ask 'if I nudge this weight slightly, how does the output change?' That question is exactly what gradient-based learning needs answered, and a step function simply can't answer it. A smooth curve can.

Everything from here builds toward that one capability: a neuron that can be trained gradually, not just corrected on mistakes.

The Sigmoid Equation

The weighted sum doesn't change. Only the final step is new:

```
z  =  w·x + b

output  =  sigma(z)  =  1 / (1 + e^-z)
```

e is Euler's number, roughly 2.718. All this formula does is squash any real number z, however large or small, smoothly into the range between 0 and 1, with no jump anywhere.

Three values worth knowing by feel

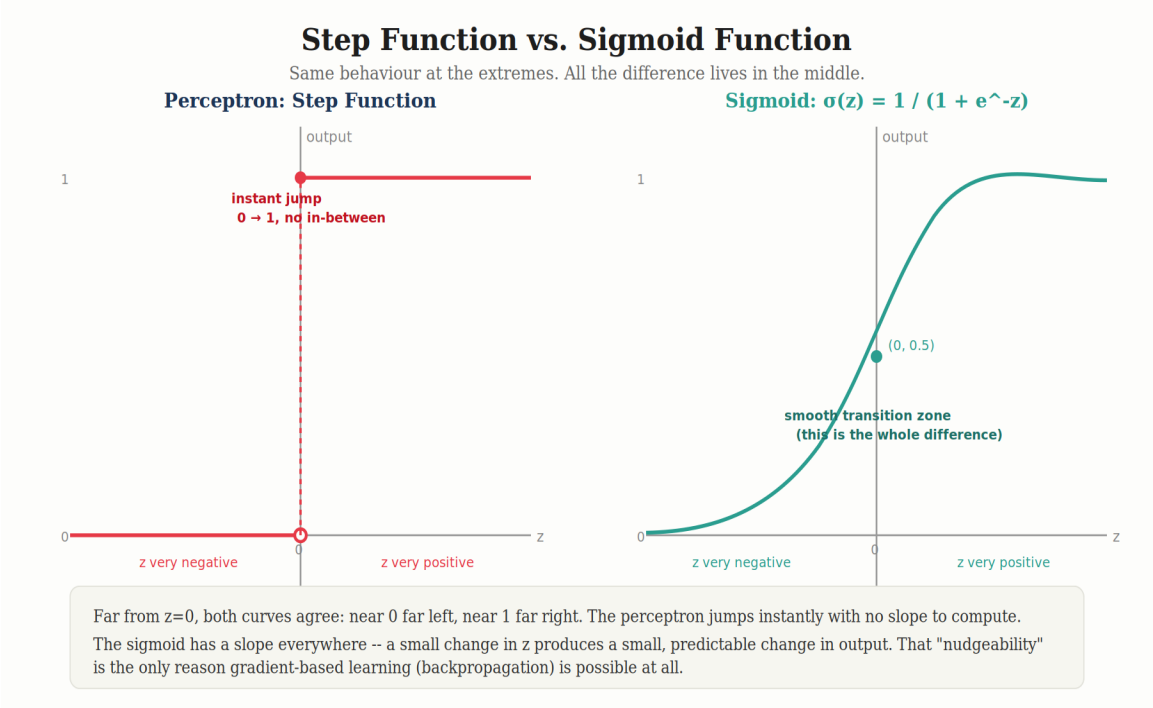
```
z = 0   →   sigma(z) = 0.500

z = 2   →   sigma(z) ≈ 0.881

z = -2  →   sigma(z) ≈ 0.119
```

z = 0 always lands exactly on 0.5 -- maximum uncertainty, the exact midpoint. As z grows positive, sigma climbs toward 1; as it falls negative, sigma drops toward 0. At the extremes this matches a perceptron exactly. The difference only shows up in the middle, which is the entire subject of the next page.

Seeing It: Step vs. Sigmoid



Both functions agree far from z=0. The perceptron jumps instantly; the sigmoid glides through a smooth transition zone instead.

On the left, the perceptron: flat at 0, then a vertical cliff straight up to 1 the instant z crosses zero. There's no 0.3 or 0.7 -- only fully off or fully on. The open and filled circles mark that the jump itself is a discontinuity, a point with no defined slope.

On the right, the sigmoid: the same 0 far to the left and 1 far to the right, but a smooth curve through the middle instead of a cliff, passing through exactly 0.5 at z = 0. That smooth middle is the entire reason this function was chosen -- not because 0.5 or the particular curve shape is special, but because every point on it has a well-defined slope, which page 15 puts to direct use.

Coding the Forward Pass

Two small methods: one for the sigmoid formula itself, one that reuses it inside the same z = w·x + b computation from Part I.

```
    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def predict(self, x):
        z = np.dot(self.weights, x) + self.bias
        return self.sigmoid(z)
```

Reading it line by line

np.exp(-z) computes e^-z directly. 1 / (1 + np.exp(-z)) is the sigmoid formula from page 12, typed exactly as written. predict() is otherwise unchanged from the perceptron -- same dot product, same bias -- except the final line calls self.sigmoid(z) instead of a hard 1-or-0 check. The return value is now a continuous number like 0.881, not just 0 or 1.

The Slope, and Why It Matters

A step function has no usable slope. Sigmoid has one everywhere, and it happens to have a clean formula written in terms of sigmoid itself:

```
sigma'(z)  =  sigma(z) × (1 - sigma(z))
```

This tells you how steep the curve is at a given point -- how much the output would shift for a tiny change in z. It peaks at 0.25 right at z = 0 and shrinks toward zero as z moves toward either extreme.

Why this matters for learning

Near the extremes the curve goes nearly flat -- the neuron is confident, close to 0 or 1, and a small weight change barely moves the output. Near the middle the curve is steepest -- the neuron is unsure, and small weight changes matter far more. The derivative captures this automatically: bigger corrections when the neuron is uncertain, smaller ones when it's already confident. A perceptron has nothing resembling this.

Coding the Derivative

```
    def sigmoid_derivative(self, z):
        s = self.sigmoid(z)
        return s * (1 - s)
```

This is the formula from page 15, typed directly. It reuses self.sigmoid(z) rather than recomputing the same thing twice -- a small habit worth keeping: if you've already written a method for something, call it instead of retyping the formula.

This method never runs during prediction. It only gets used during training, on page 18, where the neuron needs to know how sensitive its output currently is before deciding how hard to correct itself.

The Gradient Descent Update Rule

The perceptron's rule only asked right or wrong. A sigmoid neuron can ask how wrong, and how confident -- and fold both into a single update:

```
                    error  =  a - y

    w  =  w - (learning_rate) × error × sigma'(z) × x

      b  =  b - (learning_rate) × error × sigma'(z)
```

a is the neuron's actual output, a number between 0 and 1. y is the correct label, 0 or 1. error = a - y is now continuous: 0.881 - 1 = -0.119 is a small, specific amount of wrongness, not just a verdict.

That error gets multiplied by sigma'(z), the confidence factor from page 15, before it's applied. One multiplication doing two jobs at once: correcting in the direction that reduces the error, and automatically scaling that correction down whenever the neuron is already confident. This is a one-neuron version of what backpropagation does across an entire network -- the same mechanic, at the smallest scale it can exist at.

Coding the Training Loop

```
    def train(self, training_inputs, labels, epochs=2000):
        for epoch in range(epochs):
            total_loss = 0
            for x, y in zip(training_inputs, labels):
                x = np.array(x)
                z = np.dot(self.weights, x) + self.bias
                a = self.sigmoid(z)
                error = a - y
                gradient = error * self.sigmoid_derivative(z)

                self.weights -= self.learning_rate * gradient * x
                self.bias -= self.learning_rate * gradient
                total_loss += 0.5 * (error ** 2)
```

Structurally this mirrors the perceptron's train() exactly -- an epoch loop wrapping an example loop. What's different is what happens per example: instead of an if-statement that only reacts to outright mistakes, every single example nudges the weights a little, scaled by both error and confidence.

total_loss adds up 0.5 × error squared across every example -- one number summarizing how wrong the neuron currently is overall. Unlike the perceptron's error count, this shrinks gradually rather than hitting zero abruptly, which is why training here needs far more epochs -- 2000, against the perceptron's 6 -- to settle down.

Learning NAND, With Real Numbers

Same NAND data as Part I. This is the actual output from running the code above, not a rounded-off approximation:

```
epoch    0  loss = 0.52000
epoch  400  loss = 0.03130
epoch  800  loss = 0.01449
epoch 1200  loss = 0.00922
epoch 1600  loss = 0.00671
epoch 1999  loss = 0.00525

learned weights: [-5.492 -5.490]   bias: 8.326

x1  x2 | raw output | thresholded | expected
 0   0 |     0.9998  |     1       |    1
 0   1 |     0.9446  |     1       |    1
 1   0 |     0.9445  |     1       |    1
 1   1 |     0.0656  |     0       |    0
```

Two things worth noticing. Loss falls steadily -- 0.52 down to 0.005 -- rather than the perceptron's abrupt converged-or-not outcome. And the raw outputs are never exactly 0 or 1: 0.9998, 0.0656. The neuron is expressing confidence, not just a verdict -- 0.9998 reads as 'almost certainly fires', 0.0656 as 'almost certainly doesn't'. A threshold of 0.5 turns that confidence back into a final answer whenever one is actually needed.

What Smoothness Doesn't Fix

It's tempting to assume a smarter activation function solves everything, including XOR. It doesn't, and understanding exactly why is worth more than the sigmoid formula itself.

A single sigmoid neuron still computes $z = w \cdot x + b$ first, and everything downstream is just a function of that one number. Whether the output leans toward 0 or toward 1 still only depends on which side of one straight boundary z lands on -- sigmoid replaces a hard edge with a soft gradient near that same boundary. The boundary itself hasn't moved, and there's still only one of it.

Train a single SigmoidNeuron on XOR's labels and here's what actually happens: loss starts at 0.534, and by epoch 400 it's stuck flat at 0.533 for the remaining 2600 epochs. The outputs tell the real story -- 0.516, 0.500, 0.484, 0.468 -- every prediction hovering right around 0.5, the point of maximum uncertainty. The neuron hasn't almost solved it; it has genuinely given up and is guessing near the midpoint on all four cases, because no single straight line, hard or soft, separates XOR's diagonal pattern.

So what did smoothness actually buy?

Not the ability to solve harder problems with one neuron -- the ability to train networks of many neurons using gradients at all. The escape from XOR still requires depth, multiple layers, exactly as it did with NAND gates back in Part I. Sigmoid is what makes training that deeper structure possible in the first place.

Where This Goes Next

Two neurons, two lectures, one shared shape: weighted sum, then a decision. What separated them was only the last step -- a hard switch versus a smooth curve -- and that one change was the difference between a model that can only be corrected and one that can actually learn by degree.

The idea that closes the remaining gap

Backpropagation. Stack sigmoid neurons into layers -- a hidden layer feeding an output layer, the same wiring shape as the NAND-gate half-adder from the perceptron series. Each neuron's derivative, from pages 15 and 16, gets chained across layers using the calculus chain rule, so an error measured at the output can be traced backward and tell every neuron in every layer exactly how to adjust. That backward chaining is where the name comes from.

The concrete next step: extend SigmoidNeuron into a small two-layer network -- two sigmoid neurons in a hidden layer, feeding one output neuron -- and train it on XOR. Watch the loss actually fall toward zero this time, where a single neuron got stuck at 0.533. That's the moment 'depth solves what smoothness alone cannot' stops being a claim you were told and becomes a result you produced yourself.

End of lecture. Full working code: perceptron.py and sigmoid_neuron.py.